SPECTRAL A-N V18: SOFTWARE ARCHITECTURE AND REPRODUCIBILITY COMPANION

ELIUTH CHAVERO JASSO

Abstract. This companion documents the software architecture, artifact schemas, hashing/provenance contracts, and reproducibility discipline of the v18 spectral certification suite (`spectral/certification_v18/`, 85 passing tests as of this writing). It records a real, reproducible CPU/GPU parity result on the confirmed hardware (RTX PRO 5000 Blackwell, 24 GB VRAM), a concrete process-global TF32 contamination hazard found and fixed, and the current gaps in continuous integration and packaging for this new track. Section 7 records what a follow-up session has since closed: real CI (verified on GitHub Actions, including a genuine setuptools CVE caught by `pip-audit`), a 416-cell hardware-measured GPU-scaling sweep answering the throughput question this version of the paper originally left open, and a frozen, mutation-tested evidence contract.

1. Repository layout

Path	Contents
`spectral/legacy/v17/`	Immutable, hash-verified legacy script + run-log copies
`spectral/certification_v18/gates.py`	Typed-state enum, fail-closed combination logic
`spectral/certification_v18/config.py`	Certification-mode contract, enforced via `validate()`
`spectral/certification_v18/model.py`	Fresh CP-law/projector geometry (not imported from legacy)
`spectral/certification_v18/blocks/`	One module + one `BLOCK_*_FINDINGS.md` per block
`spectral/certification_v18/hardware/`	Certification-mode setup, resumable job queue
`spectral/certification_v18/artifacts/`	Saved experiment result JSON
`spectral/certification_v18/tests/`	85 tests, `pytest spectral/certification_v18/tests`

This track is deliberately NOT reimported into `src/seion_core`: it is scope class `SPECTRAL_LEGACY_TRACK` (`claims/scope_registry_v4.yaml`), evidentially separate from `CANONICAL_FINITE_CORE` by design.

2. Why `model.py` is a fresh reimplementation, not an import

The legacy script sets `torch.backends.cuda.matmul.allow_tf32 = True` and `torch.set_float32_matmul_precision("high")` unconditionally at import time when CUDA is available. These are process-global flags, not scoped to the importing module. A certification-mode process that imported the legacy script for convenience would have its TF32 discipline silently overridden by an unrelated import. 'model.py' reimplements the needed primitives (CP product, projector, associator, reduced curvature) independently, with its own tests (`test_model.py`) checked against the legacy formulas by direct line-number citation, and `hardware/certification_mode.py` provides `enter_certification_mode()` / `assert_certification_discipline()` as the single, idempotent, re-assertable enforcement point.

Date: 30 July 2026.

Companion to "A Fail-Closed Certification Framework for Cyclic N-Ary Laws, Projectors, and Multiresolution Tensor Structure" (`papers/a_to_n_certification_v18`). Documents interfaces and evidence contracts; makes no independent mathematical claim.

3. Typed-gate schema

`gates.py` defines `TypedStatus` (10 states) and `CRITICAL_GATES` (8 gates, each a tuple of contributing block names). `combine_gate_status` takes the minimum over a gate's blocks (excluding `NOT_APPLICABLE`, which is tracked separately and never silently counted as passing). `assign_block_status` is the single enforcement point preventing a screening-mode run from being assigned a certificate-tier status: it raises `ScreeningCertificateViolation` rather than downgrading silently.

4. CPU/GPU parity: a real, executed result

On the confirmed hardware (Intel Core Ultra 9 285HX, 24 logical cores, 128 GB RAM; NVIDIA RTX PRO 5000 Blackwell Laptop GPU, 24 GB VRAM, CUDA 12.8, driver 573.49), we constructed a `SpectralModelV18` instance on CPU (float64, seed 0), transferred its exact parameter/buffer values to an independently-constructed CUDA instance via `state_dict` (not `.to(device)`, which does not update the plain Python `device`/`dtype` string attributes stored on `CyclicCPProduct` — a second real hazard found during this exercise, documented in `model.py`), and computed Block A/B quantities on both devices with TF32 explicitly disabled and deterministic algorithms enabled. Results:

Quantity	CPU	CUDA
idempotence residual	1.312×10^{-15}	1.198×10^{-15}
self-adjointness residual	0.0 (exact)	2.220×10^{-16}
comm. unexplained-rel (random init)	1.0000380759060201	1.0000380759060197
wall time	19.1 ms	835.1 ms

The unexplained-rel figure (the only quantity here large enough for a relative-difference comparison to be meaningful) agrees to 4.4×10^{-16} relative difference — CPU/GPU parity confirmed at float64 for this computation. The wall-time comparison is an honest negative result for accelerating problems at this scale: GPU kernel-launch overhead dominates for $n = 24$, and CPU is $\sim 44\times$ faster in this single case. Whether that reverses once ambient dimensions are large enough to amortize the overhead was an open question at the time this result was first recorded; Section 7 reports it has since been measured directly across $n \in \{12, 24, 48, 96\}$, and it does not reverse anywhere in that range.

5. Resumable job queue

`hardware/job_queue.py` implements an append-only, JSONL-backed ledger keyed by `scientific_instance_id` (the mathematical configuration) versus `execution_id` (one specific attempt), recording seed, precision, hardware, a config hash, a script hash, checkpoint lineage (parent `execution_id`), status, retry count, and output hashes. History is never mutated, only appended to, matching the pattern already established for the separate `CANONICAL_FINITE_CORE` track's `src/seion_core/governance/runs.py` (different schema, same non-destructive philosophy, since this track's artifact set does not match that track's `REQUIRED_RUN_ARTIFACTS` contract). Tested: submit/complete, retry-increments-count while preserving prior history, and checkpoint-lineage traversal.

6. Legacy provenance

`spectral/certification_v18/tools/ingest_legacy.py` hashes both named legacy files (sha256, verified after copy), hashes all 18 on-disk run directories, parses the 9-run text log, cross-references it against on-disk `summary.json` files, reconstructs resume lineage, and reclassifies every historical run against the v18 typed-gate vocabulary.

Output: `spectral/legacy/v17/legacy_a_to_n_manifest.yaml`, `legacy_run_lineage.json`, `legacy_run_dedup_report.md`, `legacy_claim_reclassification.yaml`.

7. What has since been closed

A follow-up session closed three of the four gaps below with real, verified work rather than by editing this list in place:

- **CI**: `.github/workflows/spectral-v18.yml` now runs all 85 tests on every push/PR (confirmed green on real GitHub Actions runs, not just locally). A separate `security-and-quality.yml` adds `pip-audit` (a genuine setuptools CVE was found and fixed this way), CycloneDX SBOM generation, license reporting, `gitleaks` secret scanning, CodeQL, and a clean sdist-install matrix on `ubuntu-latest` and `windows-latest` — all verified against real CI runs, not assumed.
- **GPU throughput scaling**: measured, not merely proposed. `closure_report`, `associator_constant_report`, and `cyclic_and_gji_report` gained an explicit `device` parameter (default `"cpu"`, fully backward compatible — the 85-test suite still passes unchanged), threaded through every `torch.randn`/`Generator` call site that previously created CPU tensors unconditionally regardless of the model's device (a real bug that would have silently broken any attempt to run these functions on CUDA). A 96-cell pilot sweep and a 320-cell broad screening sweep (416 cells total, 0 failures, $\approx$ 121 minutes combined wall time) found GPU consistently 3.2×–3.5× slower than CPU across $n \in \{12, 24, 48, 96\}$ — a genuine, hardware- measured negative result, stable across an 8× range in ambient dimension, not a small-scale artifact. Full detail in the companion certification paper's Section 4.
- **Evidence-contract freeze**: `governance/RELEASE_GATES.yaml` v5 registers the 4 critical gates the original 8-gate taxonomy didn't cover (`construction_integrity`, `novelty`, `paper`, `release`); a new `scientific_instance.schema.json` formalizes the `scientific_instance_id`/`execution_id`/ `optimizer_restart_id` identity hierarchy this paper's job-queue section describes informally; 7 invariants (bound ordering, exact-status- requires-zero-gap, screening-cannot-certify, etc.) are now enforced as real code (`src/seion_core/governance/evidence_contract.py`) with 23 passing mutation tests, plus a sha256-hash-pinned schema-drift detector, hand-verified to actually catch tampering (a schema file was deliberately edited, the drift test confirmed to fail, then restored and confirmed to pass again).
- **Not yet closed**: clean-environment reproduction (a fresh virtualenv/container install-and-rerun of the full v18 suite specifically, as opposed to the repository's general clean-install CI matrix, which does exercise a fresh sdist install but not this track's own 85-test run inside it) has still not been exercised as a dedicated pass.

Independent Researcher, Apizaco, Tlaxcala, Mexico